#26_Walk-through_Rock_Paper_Scissors

17/06/2025

Game flow:

1. The user makes a choice
2. The computer makes a choice
3. The winner is displayed.

Pseudocode (INFORMAL)

Given a collection containing rock, paper and scissors strings

- ☐ Ask the user to choose one.
  - → save to user_choice
- ☐ Implement a random.choice for the computer
  - → save to computer_choice
- ☐ Compare the two choices
  - → if user == rock and computer == scissors
    - -User wins
  - if user == scissors and computer == paper
    - -User wins
  - if user == paper and computer == rock
    - -User wins
  - → repeat the if statements above with computer winning
  - → else it's a tie.
- ☐ Ask the user if they would like to play again

Formal Pseudocode

14/06/2025

```
START
# Given a collection of strings: 'rock', 'paper', 'scissors'
    assigned to constant "VALID_CHOICES"

PROCEDURE FUNCTION defined as: prompt(message)
# Provide a clear indication of the outputs from the program
PRINT(" => {message}")
END FUNCTION
    PROCEDURE FUNCTION defined as display_winner(player, computer)
# Outputs the winner based on the player and computer
    choices
    IF (player chooses 'rock' AND computer chooses 'scissors') OR
    (player chooses 'scissors' AND computer chooses 'paper') OR
    (player chooses 'paper' AND computer chooses 'rock'):
    CALL prompt("you win")

    ELIF (same as IF but swapping player and computer)
    CALL prompt("computer wins")

    ELSE
    CALL prompt("it's a tie")

    END IF
END FUNCTION

WHILE True
    CALL Prompt("Choose a move")
    GET user choice
    SET user choice to user_choice

    WHILE user_choice not in VALID_CHOICES
```

```
    CALL prompt("choice not valid, try again")
    SET user_choice = user input
ENDWHILE

SET computer_choice = random value from VALID_CHOICES

CALL prompt("Your choice" + user_choice + "computer
        choice" + computer_choice + ".")

CALL display_winner(user_choice, computer_choice)

WHILE True
    CALL prompt("ask user to play again? (y/n)")
    SET answer = user input
    IF answer starts with 'y' or 'n':
        BREAK
    CAL prompt("Enter a valid response.")

IF answer[0] == 'n':
    BREAK

END WHILE

END PROGRAM
```

14/06/2025

Starting to Code

- activate environment that has pylint installed
- create file name rock_paper_scissors.py
- implement prompt function to standardise outputs

```
def prompt(message):
    print(f"==> {message}")
```

Write code that asks user to choose one of: rock, paper or scissors. Add the code below the prompt function.

```
prompt('Choose one: rock, paper, scissors')
choice = input()

while choice not in ['rock', 'paper', 'scissors']:
    prompt("That's not a valid choice")
    choice = input
```

better to assign this list to a constant to make it more readable

Extract the list ['rock', 'paper', 'scissors'] out and assign it to a constant.

```
VALID_CHOICES = ['rock', 'paper', 'scissors']
```

NB: SCREAMING_SNAKE_CASE for constants.

This should be inserted at the top of the program.

Now we can use VALID_CHOICES rather than writing out the list whenever requiring the elements.

14/06/2025

Amending the code to include the VALID_CHOICES

```
prompt('Choose one: rock, paper, scissors')
choice = input()
while choice not in VALID_CHOICES:
    prompt("That's not a valid choice")
    choice = input()
```

Could also use VALID_CHOICES here aswell

An advantage of assigning the list to a constant, it allows us to add more choices to the list without having to add those choices to every section of code that relies on the list.

```
prompt(f"Choose one: {", ".join(VALID_CHOICES)}")
choice = input()
```

delimiter

String interpolation and join method used to display the options for the player to choose from.

The join method concatenates the list of strings. Without the delimiter ", ", the concatenation would appears as rockpaperscissors. Instead it appears as rock, paper, scissors.

space after ,

- Making a choice for the computer we can use the random.choice method. Firstly, need to import the random module as random.choice is not automatically available in Python. choice() method returns a randomly selected element from the sequence.

14/06/2025

```
import random

# code omitted

prompt(f'Choose one: {", ".join(VALID_CHOICES)}')
choice = input()

while choice not in VALID_CHOICES:
    prompt("That's not a valid choice")
    choice = input()

computer_choice = random.choice(VALID_CHOICES)
```

Now, inform the user of what the two choices are:

```
# code omitted

computer_choice = random.choice(VALID_CHOICES)

prompt(f"You chose {choice}, computer chose {computer_choice}")
```

Now, time to determine who the winner is:

```
prompt(f"You chose {choice}, computer chose {computer_choice}")

if ((choice == "rock" and computer_choice == "scissors") or
    (choice == "scissors" and computer_choice == "paper") or
    (choice == "paper" and computer_choice == "rock")):
    prompt("You win")
```

14/06/2025

```
elif ((choice == "rock" and computer_choice == "paper") or
    (choice == "paper" and computer_choice == "scissors") or
    (choice == "scissors" and computer_choice == "rock")
    prompt("Computer wins!")

else:
    prompt("It's a tie!")
```

All that is needed to be added is providing the user with an option to play again. Below is the full code

```
import random

VALID_CHOICES = [rock, paper, scissors]

def prompt(message):
    print(f"==> {message}")

while True:
    prompt(f'Choose one: {f", ".join{VALID_CHOICES}"}')
    choice = input()

    while choice not in VALID_CHOICES:
        prompt("That's not a valid choice.")
        choice = input()

    computer_choice = random.choice(VALID_CHOICES)

    prompt(f"You chose {choice}, computer chose {computer_choice}")
```

Outer while loop

This can be extracted out and incorporated into a function

14/06/2025

```
if ((choice == "rock" and computer_choice == "scissors") or
    (choice == "scissors" and computer_choice == "paper") or
    (choice == "paper" and computer_choice == "rock"):
    prompt ("You win!")
elif ((choice=="rock" and computer_choice == "paper") or
    (choice == "paper" and computer_choice == "scissors") or
    (choice == "scissors" and computer_choice == "rock"):
    prompt ("Computer wins!")
else:
    prompt ("It's a tie")

while True:                    ← inner while loop
    prompt("Do you want to play again (y/n)? ")
    answer = input().lower()

    if answer.startswith('y') or answer.startswith('n'):
        break                  ← breaks out of inner while loop
    else:
        prompt("That's not a valid choice")

    if answer[0] == 'n':
        break                  ← breaks out of outer while loop
```

`while True` loops continue until a `break` statement is encountered by the program.

14/06/2025.

Extracting the if statements into a function: Insert function below prompt function (or above, it does not affect the program).

```
def display_winner(player,computer):
    prompt(f"You chose {choice}, computer chose {computer_choice}")

    if ((player == "rock" and computer == "scissors") or
    elif :
    else:
        prompt("It's a tie!")
```

Insert display_winner call below the declaration of computer_choice

```
computer_choice = random.choice(VALID_CHOICES)

display_winner(choice, computer_choice)
```

FINAL CODE.

```
import random

VALID_CHOICES = ["rock", "paper", "scissors"]

def prompt(message)
    print(f"==>{message}")

def display_winner(player, computer)
    prompt(f"You chose {player}, computer chose {computer}")

    if ((player == "rock" and computer == "scissors") or
```

```
    (player == "paper" and computer == "rock") or
    (player == "scissors" and computer == "paper")):
        prompt("You win!")
    elif ((player == "rock" and computer == "paper") or
        (player == "paper" and computer == "scissors") or
        (player == "scissor" and computer == "rock"):
        prompt("Computer wins!")
    else:
        prompt("It's a tie!")

while True
    prompt(f'Choose one: {", ".join(VALID_CHOICES)}')
    choice = input()

    while choice not in VALID_CHOICES:
        prompt("That's not a valid choice.")
        choice = input()

    computer_choice = random.choice(VALID_CHOICES)

    display_winner(choice, computer_choice)

    while True:
        prompt("Do you want to play again (y/n)?")
        answer = input().lower()

        if answer.startswith('n') or answer.startswith('y'):
            break
        else:
            prompt("That's not a valid choice.")

    if answer[0] == 'n':
        break
```

14/06/2025

Implementing the display_winner function allows for greater readability of the main section of code and also allows for easier maintainability.

Things to Consider

1. display_winner function can be either below (as we did) or above the prompt function despite the fact that display_winner invokes the prompt function. As long as all functions are defined prior to being invoked. i.e., prompt function is not invoked by the display_winner function until the main code runs and invokes the display_winner function.

2. If the intention of the display_winner function is to return a string, opposed to outputting the string directly.
   - Would implement a return string to the display_winner function rather than invoking the prompt message
   - Invoke the display_winner function within print function.

```
print(display_winner(choice, computer_choice))
```

3. We used a while True loop for asking the user if they would like to play again. Alternatively, we could use a conditional while loop.

```
answer = ''

while not (answer.startswith('y') or answer.startswith('n')):
    prompt("Would you like to play again (y/n)?")
    answer = input().lower()
    if not (answer.startswith('y') or answer.startswith('n')):
        prompt("Provide a valid response.")
```

14/06/2025

NB: Using startswith method allows for checking an empty string, whereas implementing answer[0] will return an error if answer = ''.

i.e.,

```
answer = ''

if answer[0] == 'n'
    print(True)     # IndexError: string index out of range.
```

```
answer = ''

if not (answer.startswith('y') or answer.startswith('n'))
    print(True)  # True
```